minimal direct impact on rendering performance for most modern sites, they can slow down style recalculation and matching in extremely large or complex UIs involving thousands or millions of DOM nodes. In such cases, overly specific selectors may become a noticeable performance bottleneck.

For this topic, are you able to answer these follow-up/probing questions?

- "How would you establish and enforce team guidelines to minimize the use of `!important` in CSS, while still ensuring that critical styles are reliably applied?"
- "How would you effectively debug a CSS specificity problem when styles are not being applied as expected?"
- "What strategies or methodologies would you recommend for organizing and structuring CSS code to prevent specificity conflicts and ensure maintainability in a large, multi-developer project?"
- "With the introduction of new CSS features like `@scope` and `:has()`, how do you envision the landscape of specificity management evolving?"
- "Does high specificity have a measurable impact on rendering performance, particularly in large or complex web applications?"

Initial question: "How do you optimize CSS for performance?"

Analysis: This question evaluates a candidate's practical knowledge of CSS performance optimization and their commitment to user experience and scalability. A strong answer demonstrates hands-on experience with large-scale applications and highlights specific strategies used to optimize CSS for performance, maintainability, and efficiency.

Answer: Optimizing CSS for performance means making stylesheets lean, fast, and efficient.

Demonstrating a strong understanding of CSS performance optimization can set a candidate apart in interviews. It is important to explain not only what each technique does, but also why it matters for user experience and scalability. During practical coding interviews, these best practices should be proactively applied—such as keeping selectors simple, avoiding unnecessary styles, using modern layout techniques, and structuring CSS for